4 장 선형 회귀 - 보험료 예측하기 (교재 실습 강의록)

교수자: Prof. Jung, Minpo **교과목**: Machine Learning **학기**: 2026 년도 1 학기 **대상**: 박사과정 (중국 박사과정 학생) **교재**: #MUSTHAVE 머신러닝 **실습 코드**: colab.research.google.com/github/musthave-ML10/notebooks/blob/main/ **실습 파일**: 04_Linear Regression.ipynb

---

강의 개요

항목	내용
소요 시간	약 75 분
학습 목표	선형 회귀 모델을 사용하여 보험료를 예측하고, 모델 평가 및 해석까지 수행
실습 환경	Google Colab
준비물	교재 4 장, Google 계정, 웹 브라우저

도입 (약 5 분)

안녕하세요, 여러분. 오늘은 머신러닝에서 가장 기초적이면서도 중요한 알고리즘인 **선형 회귀 (Linear Regression)** 를 배우겠습니다.

지금 바로 Google Colab 을 열어주세요. 교재에서 제공하는 노트북 파일을 사용할 겁니다. 주소창에 다음을 입력해서 실습 파일을 열어주세요.

colab.research.google.com/github/musthave-ML10/notebooks/blob/main/

여기서 **04_Linear Regression.ipynb** 파일을 선택하면 됩니다. 모두 열었나요? 그러면 시작하겠습니다.

선형 회귀 소개 (약 8 분)

선형 회귀란 무엇인가?

선형 회귀는 **가장 기초적인 머신러닝 모델**입니다. 이름에서 알 수 있듯이, 데이터의 관계를 **직선 (선형)** 으로 표현하여 예측하는 방법입니다.

여기서 핵심은 **연속형 목표 변수**를 예측한다는 점입니다. 예를 들어, 키를 예측한다면 165.5cm, 172.3cm 처럼 연속적인 숫자가 나옵니다. "크다/작다"처럼 범주로 나누는 것이 아니라, 구체적인 **숫자 값**을 예측하는 것이죠.

왜 TOP 10 에 선정되었는가?

교재에서 선형 회귀를 TOP 10 알고리즘에 포함시킨 이유는, 이것이 **기초 알고리즘**이자 **베이스라인 (baseline)** 역할을 하기 때문입니다. 베이스라인이란"기본 성능의 기준점"을 의미합니다. 어떤 복잡한 모델을 만들었을 때, "선형 회귀보다 얼마나 좋은가?"를 비교하는 출발점이 되는 것입니다.

장단점 정리

장점	단점
모델이 간단하다	예측력이 상대적으로 낮다
해석이 쉽다 (계수로 설명 가능)	독립변수와 종속변수 간 선형 관계를 전제로 한다
학습 속도가 빠르다	이상치 (outlier) 에 민감하다

장점부터 보겠습니다. 모델이 간단하다는 것은 코드 몇 줄이면 만들 수 있다는 뜻이고, 해석이 쉽다는 것은"이 변수가 1 증가하면 결과가 얼마나 변하는지"를 계수 (coefficient) 로 바로 설명할 수 있다는 뜻입니다. 이것은 실무에서 매우 중요합니다. 경영진에게"왜 이런 예측이 나왔나요?"라고 질문받을 때 명확하게 답할 수 있습니다.

단점은 예측력이 상대적으로 낮다는 점입니다. 복잡한 비선형 관계를 포착하지 못합니다. 그리고 독립변수와 종속변수 사이에 선형 관계가 있다는 것을 전제로 합니다. 실제 데이터가 곡선 관계를 가지면 성능이 떨어집니다.

어디에 유용한가?

선형 회귀는 **연속형 변수를 예측**하는 곳이라면 어디든 사용할 수 있습니다.

- **BMI(체질량지수)** 예측
- **매출액** 예측
- **전력 사용량** 예측
- **보험료** 예측 (오늘 우리가 할 것!)

이런 곳에서 특히 유용합니다. 오늘은 보험료 예측을 직접 해보겠습니다.

4.1 문제 정의: 한눈에 보는 예측 목표 (약 3 분)

자, 이제 교재의 4.1 절을 펴주세요.

미션

> **보험 데이터셋을 이용하여 보험사에서 청구할 보험료를 예측하라.**

이것이 오늘의 미션입니다. 표로 정리하면 다음과 같습니다.

항목	내용
난이도	(1 단계)
알고리즘	Linear Regression (선형 회귀)
데이터셋	insurance.csv
종속변수 (타깃)	charges (청구비용)
문제 유형	회귀 (Regression)
평가지표	RMSE
사용할 모델	LinearRegression

여기서 중요한 것 두 가지를 짚겠습니다.

첫째, **문제 유형이"회귀"**입니다. 회귀 (Regression) 는 연속적인 숫자를 예측하는 문제입니다. "분류 (Classification)"와 대비됩니다. 분류는"A 인가 B 인가"를 맞추는 것이고, 회귀는"구체적인 숫자가 얼마인가"를 맞추는 것입니다.

둘째, **평가지표가 RMSE** 입니다. RMSE 는 Root Mean Squared Error, 즉 평균 제곱근 오차입니다. 나중에 자세히 설명하겠지만, 쉽게 말해"예측값과 실제값의 차이를 하나의 숫자로 요약한 것"입니다. 이 값이 작을수록 예측을 잘한 것입니다.

사용할 라이브러리 버전

라이브러리	버전
numpy	1.19.5
pandas	1.3.5
seaborn	0.11.2
matplotlib	3.2.2
sklearn	1.0.2

Colab 에서는 이 라이브러리들이 이미 설치되어 있으니 별도 설치는 필요 없습니다. 바로 실습을 시작하겠습니다.

4.2 라이브러리 및 데이터 불러오기 (약 5 분)

자, 이제 코드를 직접 작성해봅시다. Colab 에서 새 셀을 만들고, 다음 코드를 **직접 타이핑**해주세요. 복사-붙여넣기 하지 말고, 직접 타이핑하면서 익히는 것이 중요합니다.

```
import pandas as pd  # 판다스 라이브러리 임포트

file_url = 'https://media.githubusercontent.com/media/musthave-ML10/data_source/main/insurance
data = pd.read_csv(file_url)  # 데이터셋 읽기
```

한 줄씩 설명하겠습니다.

첫 번째 줄: `import pandas as pd` 는 판다스 라이브러리를 불러오는 코드입니다. `as pd` 는 별칭 (alias) 을 pd 로 지정하겠다는 뜻입니다. 앞으로 pandas 대신 pd 라고 짧게 쓸 수 있습니다.

두 번째 줄: `file_url` 에 데이터 파일의 URL 주소를 저장합니다. 이 데이터는 교재에서 GitHub 에 올려놓은 보험 데이터입니다.

세 번째 줄: `pd.read_csv(file_url)` 은 CSV 파일을 읽어서 판다스 DataFrame 으로 변환하는 함수입니다. 결과를 `data` 라는 변수에 저장합니다.

Shift+Enter 를 눌러서 실행해주세요. 오류 없이 실행되면 성공입니다.

> **참고**: 만약 URL 에서 데이터를 못 불러온다면, 인터넷 연결을 확인해주세요. Colab 은 인터넷에 연결되어 있어야 합니다.

4.3 데이터 확인하기 (약 10 분)

데이터를 불러왔으면, 가장 먼저 해야 할 일은 **데이터가 어떻게 생겼는지 확인하는 것**입니다. 이것은 매우 중요한 습관입니다. 어떤 머신러닝 프로젝트를 하든, 데이터를 먼저 살펴보는 것이 첫 번째 단계입니다.

전체 데이터 출력

새 셀에 다음을 입력하고 실행하세요.

```
data  # 전체 데이터 출력
```

실행하면 **1338 rows x 6 columns** 라고 표시될 것입니다. 데이터가 1338 행, 6 열이라는 뜻입니다.

변수 (컬럼) 6 개를 하나씩 살펴봅시다.

변수명	설명	유형
age	연령	연속형
sex	성별 (1= 남자, 0= 여자)	범주형
bmi	체질량지수	연속형
children	자녀 수	연속형
smoker	흡연 여부 (1= 흡연자, 0= 비흡연자)	범주형
charges	청구 비용 (종속변수)	연속형

인덱스는 0 번부터 1337 번까지, 총 1338 개입니다.

상위 5 줄 확인

```
data.head()  # 상위 5 줄 출력
```

`head()` 함수는 데이터의 처음 5 줄만 보여줍니다. 데이터가 크면 전체를 출력하기 어렵기 때문에, 이렇게 일부만 확인하는 것이 일반적입니다.

여기서 주의해서 볼 점이 있습니다. `sex` 와 `smoker` 변수를 보세요. 원래 성별은"male/female", 흡연 여부는 "yes/no"로 되어 있어야 하는데, 여기서는 이미 **숫자로 인코딩 (encoding)** 되어 있습니다.

- **sex**: 1 = 남자 (male), 0 = 여자 (female)
- **smoker**: 1 = 흡연자 (yes), 0 = 비흡연자 (no)

이것이 중요한 이유가 뭘까요? 머신러닝 모델은 **숫자**만 처리할 수 있기 때문입니다. "male", "female"같은 문자 (text) 는 그대로 넣을 수 없습니다. 그래서 문자를 숫자로 변환하는"인코딩"과정이 필요한데, 이 데이터셋에서는 이미 처리가 되어 있는 것입니다.

연속형 변수와 범주형 변수

잠깐, 여기서 중요한 개념을 정리하겠습니다.

연속형 변수 (Continuous Variable): 연속적인 숫자 값을 가지는 변수입니다. age(18, 19, 20...), bmi(27.9, 33.7...), charges(16884.92...) 등이 해당됩니다. 이 값들은 사칙연산이 의미가 있습니다. "평균 나이 39 세"라고 하면 말이 됩니다.

범주형 변수 (Categorical Variable): 범주 (카테고리) 를 나타내는 변수입니다. sex(남/여), smoker(흡연/비흡연) 등이 해당됩니다. 숫자로 인코딩되어 있어도 본질은 범주형입니다. "평균 성별 0.5"라고 하면 말이 안 됩니다.

이 구분은 나중에 더 복잡한 모델을 다룰 때 매우 중요해집니다.

컬럼 정보 확인

```
data.info()  # 컬럼 정보 출력
```

실행하면 다음과 같은 정보를 볼 수 있습니다.

- **Column**: 컬럼 (변수) 이름
- **Non-Null Count**: 결측치가 아닌 데이터의 개수
- **Dtype**: 데이터 타입 (int64: 정수, float64: 실수)

여기서 가장 중요한 것은 **Non-Null Count** 입니다. 모든 컬럼이 1338 non-null 로 나온다면, **결측치 (missing value) 가 없다**는 뜻입니다. 결측치가 없으니 별도의 결측치 처리 과정이 필요 없습니다. 실제 데이터에서는 결측치가 있는 경우가 매우 많으니, 항상 이 부분을 확인하는 습관을 들이세요.

통계 정보 확인

```
data.describe()  # 통계 정보 출력
```

그런데 소수점이 너무 많이 나와서 보기 어렵죠? `round()` 함수를 사용해서 정리합시다.

```
round(data.describe(), 2)  # 소수점 2 째자리까지 반올림
```

이제 각 통계량을 설명하겠습니다.

통계량	의미	예시 (age)
count	데이터 개수	1338
mean	평균	39.21
std	표준편차	14.05
min	최솟값	18
25%	1 사분위수	27
50%	2 사분위수 (중앙값)	39
75%	3 사분위수	51
max	최댓값	64

사분위수 (Quantile) 개념을 설명하겠습니다. 데이터를 크기 순서로 정렬했을 때, 4 등분하는 지점의 값입니다.

- **25% (Q1, 1 사분위수)**: 하위 25% 지점의 값입니다. age 의 경우 27 세입니다.
- **50% (Q2, 2 사분위수, 중앙값)**: 정확히 가운데 지점의 값입니다. age 의 경우 39 세입니다.
- **75% (Q3, 3 사분위수)**: 하위 75% 지점의 값입니다. age 의 경우 51 세입니다.

사분위수는 데이터의 분포를 파악하는 데 매우 유용합니다. 예를 들어, charges(청구비용) 의 경우 mean(평균) 이 약 13270 인데 50%(중앙값) 는 약 9382 입니다. 평균이 중앙값보다 훨씬 크다는 것은 **오른쪽으로 치우친 분포 (right-skewed)** 라는 뜻입니다. 즉, 대부분의 사람은 보험료가 낮지만, 일부 사람의 보험료가 매우 높아서 평균을 끌어올리고 있는 것입니다.

4.4 전처리: 학습셋과 시험셋 나누기 (약 10 분)

4.4.1 변수와 데이터셋을 나누는 이유

데이터를 확인했으니, 이제 **전처리 (Preprocessing)** 단계입니다. 오늘 할 전처리는 데이터를 나누는 것입니다.

두 가지를 나눠야 합니다.

첫째, 독립변수와 종속변수를 나눕니다.

- **독립변수 (Independent Variable, Feature)**: 예측에 사용하는 변수들입니다. age, sex, bmi, children, smoker 가 독립변수입니다.
- **종속변수 (Dependent Variable, Target)**: 예측하고 싶은 변수입니다. charges 가 종속변수입니다.

왜 나누는가? 모델에게"이 정보 (독립변수) 를 가지고 이것 (종속변수) 을 예측해라"라고 알려줘야 하기 때문입니다. 통상적으로 독립변수는 **X**, 종속변수는 **y** 로 표기합니다.

둘째, 학습셋 (Train Set) 과 시험셋 (Test Set) 을 나눕니다.

이것은 매우 중요한 개념입니다. 비유를 들어보겠습니다.

시험을 준비한다고 합시다. 문제집으로 공부 (학습) 를 하고, 실제 시험 (테스트) 을 봅니다. 만약 시험 문제가 문제집과 똑같다면? 100 점을 받겠지만, 그건 실력이 아닙니다. 진짜 실력을 평가하려면 **처음 보는 문제**로 시험을 봐야 합니다.

머신러닝도 마찬가지입니다. 모델이 학습에 사용한 데이터로 평가하면 성능이 좋게 나오지만, 그건 실제 성능이 아닙니다. 이것을 **오버피팅 (Overfitting, 과적합)** 이라고 합니다. 학습 데이터에만 지나치게 맞춰져서 새로운 데이터에는 성능이 떨어지는 현상입니다.

이를 방지하기 위해 데이터를 **학습셋**과 **시험셋**으로 나눕니다.

- **학습셋 (Train Set)**: 모델이 학습 (공부) 하는 데이터
- **시험셋 (Test Set)**: 모델의 성능을 평가 (시험) 하는 데이터

비율은 보통 **7:3** 또는 **8:2** 를 사용합니다. 오늘은 8:2 로 나누겠습니다.

4.4.2 데이터셋 나누기

자, 코드를 작성합시다. 새 셀에 다음을 입력하세요.

```
X = data[['age', 'sex', 'bmi', 'children', 'smoker']]  # 독립변수
y = data['charges']  # 종속변수
```

첫 줄에서 `data[['age', 'sex', 'bmi', 'children', 'smoker']]` 는 data 에서 이 5 개 컬럼만 선택하는 것입니다. 대괄호가 두 개인 이유는, 여러 컬럼을 동시에 선택할 때는 리스트 형태로 넘겨야 하기 때문입니다. 결과를 X 에 저장합니다.

두 번째 줄에서 `data['charges']` 는 charges 컬럼 하나만 선택하는 것입니다. 대괄호가 하나입니다. 결과를 y 에 저장합니다.

이제 학습셋과 시험셋으로 나눕니다. 새 셀에 다음을 입력하세요.

```
from sklearn.model_selection import train_test_split  # 사이킷런 임포트
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=100)  #
```

한 줄씩 설명하겠습니다.

`from sklearn.model_selection import train_test_split`: 사이킷런 (scikit-learn) 라이브러리에서 `train_test_split` 함수를 가져옵니다. 이 함수가 데이터를 자동으로 나눠줍니다.

`train_test_split(X, y, test_size=0.2, random_state=100)`: 핵심 매개변수를 보겠습니다.

- **X, y**: 나눌 데이터입니다. 독립변수와 종속변수를 함께 넣어줍니다.
- **test_size=0.2**: 시험셋의 비율입니다. 0.2 는 20% 를 의미합니다. 즉, 80% 는 학습용, 20% 는 시험용입니다.
- **random_state=100**: 랜덤 시드를 고정합니다. 데이터를 나눌 때 무작위로 섞는데, 이 시드를 고정하면 **항상 같은 결과**가 나옵니다. 재현성 (reproducibility) 을 위해 중요합니다. 여러분이 제가 보여주는 것과 같은 결과를 얻으려면 반드시 random_state=100 을 사용해야 합니다.

결과로 **4 개의 데이터셋**이 만들어집니다.

데이터셋	내용	크기
X_train	학습용 독립변수	1070 행 x 5 열
X_test	시험용 독립변수	268 행 x 5 열
y_train	학습용 종속변수	1070 개
y_test	시험용 종속변수	268 개

1338 개 데이터 중 80% 인 1070 개가 학습용, 20% 인 268 개가 시험용으로 나뉩니다.

Shift+Enter 로 실행해주세요. 오류가 없으면 성공입니다.

4.5 모델링 (약 5 분)

드디어 모델을 만들 차례입니다. 놀라울 정도로 간단합니다. 새 셀에 다음 코드를 입력하세요.

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
```

이 세 줄이 머신러닝 모델링의 핵심 패턴입니다. 이 패턴은 거의 모든 사이킷런 모델에서 동일합니다. 하나씩 보겠습니다.

1 단계 - 알고리즘 선택 (import)

```
from sklearn.linear_model import LinearRegression
```

사이킷런에서 LinearRegression 알고리즘을 가져옵니다.

2 단계 - 모델 생성 (객체 생성)

```
model = LinearRegression()
```

LinearRegression 모델 객체를 만들어서 `model` 이라는 변수에 저장합니다. 이 시점에서는 아직 아무것도 학습하지 않은"빈 모델"입니다.

3 단계 - 모델 학습 (fit)

```
model.fit(X_train, y_train)
```

`fit()` 함수가 실제 학습을 수행합니다. **학습용 독립변수 (X_train)** 와 **학습용 종속변수 (y_train)** 를 넣어줍니다. 모델은 이 데이터를 보고"age, sex, bmi, children, smoker 와 charges 사이의 관계"를 학습합니다.

> **중요**: fit() 에는 반드시 **학습셋 (train)** 을 넣어야 합니다. 시험셋 (test) 을 넣으면 안 됩니다!

실행하면 `LinearRegression()` 이라는 출력이 나옵니다. 이것은 모델이 성공적으로 학습되었다는 뜻입니다.

이 세 줄의 패턴 **import → 객체 생성 → fit** 을 꼭 기억하세요. 앞으로 다른 알고리즘을 배울 때도 이 패턴은 동일합니다. LinearRegression 자리에 다른 알고리즘 이름만 바꾸면 됩니다.

4.6 모델을 활용해 예측하기 (약 3 분)

모델이 학습되었으니, 이제 이 모델로 예측을 해봅시다. 새 셀에 다음을 입력하세요.

```
pred = model.predict(X_test)
```

`predict()` 함수에 **시험용 독립변수 (X_test)** 를 넣어줍니다. 모델은 이 데이터를 보고 charges 를 예측합니다. 예측 결과를 `pred` 라는 변수에 저장합니다.

여기서 매우 중요한 점 하나! **종속변수 (y_test) 는 넣으면 안 됩니다.** predict() 에는 독립변수만 넣어야 합니다. 모델에게"정답 (charges)"을 보여주면 안 되는 것이죠. 마치 시험을 볼 때 답안지를 보면서 풀면 안 되는 것과 같습니다.

그러면 y_test 는 언제 사용하나요? **평가 단계**에서 사용합니다. 모델의 예측값 (pred) 과 실제 정답 (y_test) 을 비교해서 모델이 얼마나 잘 예측했는지 평가할 때 사용합니다.

실행하세요. 오류가 없으면 예측이 완료된 것입니다.

4.7 예측 모델 평가하기 (약 18 분)

모델이 예측을 했으니, 이제"얼마나 잘 예측했는가?"를 평가해야 합니다. 교재에서는 세 가지 방법을 제시합니다.

4.7.1 테이블로 평가하기

가장 직관적인 방법입니다. 실제값과 예측값을 나란히 놓고 비교합니다. 새 셀에 다음을 입력하세요.

```
comparison = pd.DataFrame({'actual': y_test, 'pred': pred})
comparison
```

`pd.DataFrame()` 으로 새로운 표를 만듭니다. 'actual'열에 실제값 (y_test), 'pred'열에 예측값 (pred) 을 넣습니다.

실행하면 **268 rows x 2 columns** 테이블이 나옵니다. 268 개의 시험 데이터에 대해 실제값과 예측값이 나란히 보입니다.

몇 개만 살펴보세요. actual 과 pred 가 비슷한 행도 있고, 많이 차이나는 행도 있을 것입니다. 그런데 268 개를 일일이 비교하기는 어렵겠죠? 그래서 그래프와 통계적 방법이 필요합니다.

4.7.2 그래프로 평가하기

산점도 (Scatter Plot) 를 그려봅시다. 새 셀에 다음을 입력하세요.

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(10,10))
sns.scatterplot(x='actual', y='pred', data=comparison)
```

matplotlib 과 seaborn 은 파이썬의 시각화 라이브러리입니다. scatterplot 은 산점도를 그리는 함수입니다. x 축에 actual(실제값), y 축에 pred(예측값) 을 넣습니다.

실행하면 산점도가 나타납니다.

이 그래프를 어떻게 해석하는지 설명하겠습니다.

만약 모든 예측이 완벽하다면, 즉 예측값 = 실제값이라면, 모든 점이 **대각선 (45 도 직선)** 위에 놓일 것입니다. 이 대각선을 상상해 보세요. 왼쪽 아래에서 오른쪽 위로 가는 직선입니다.

이 대각선을 기준으로 세 영역을 구분할 수 있습니다.

- **영역 1 (대각선 위 근처)**: 정확한 예측. 실제값과 예측값이 비슷합니다.
- **영역 2 (대각선 위쪽)**: 과대예측 (Overestimation). 실제보다 높게 예측했습니다.
- **영역 3 (대각선 아래쪽)**: 과소예측 (Underestimation). 실제보다 낮게 예측했습니다.

산점도를 보면, 대각선 근처에 몰려있는 점들도 있지만, 상당히 벗어나 있는 점들도 있을 것입니다. 특히 실제 보험료가 높은 쪽에서 과소예측이 많이 보일 것입니다. 이것은 선형 회귀 모델의 한계를 보여줍니다.

4.7.3 통계적인 방법으로 평가하기: RMSE

그래프는 직관적이지만, "정확히 얼마나 좋은가?"를 숫자 하나로 표현하기 어렵습니다. 그래서 통계적 지표가 필요합니다.

오차 (Error) 의 문제

오차 = 예측값 - 실제값입니다. 예를 들어, - 실제값 100, 예측값 120 이면 오차 = +20 - 실제값 100, 예측값 80 이면 오차 = -20

이 두 오차를 더하면 0 이 됩니다. 양수와 음수가 상쇄되기 때문입니다. 이것은 문제가 있습니다. 실제로는 두 번 다 20 만큼 틀렸는데, 합계는 0 이 나오니까요. 그래서 오차의 부호를 없애는 방법이 필요합니다.

두 가지 해결책:

1. **절대값을 씌운다** → MAE (Mean Absolute Error, 평균 절대 오차)
2. **제곱을 한다** → MSE (Mean Squared Error, 평균 제곱 오차)

교재에서는 Table A 와 Table B 예시를 통해 MAE 와 MSE 의 차이를 설명합니다.

- **Table A**: 오차가 고르게 분포 (1, 2, 1, 2)
- **Table B**: 오차가 한 곳에 집중 (0, 0, 0, 6)

MAE 는 두 테이블 모두 1.5 로 동일합니다. 하지만 MSE 는 Table B 가 더 큽니다. 왜냐하면 MSE 는 큰 오차에 **제곱으로 패널티**를 주기 때문입니다. 6 이라는 큰 오차는 제곱하면 36 이 됩니다.

따라서 MSE 는 **큰 오차에 민감**합니다. 이런 특성 때문에 많은 경우에 MSE 가 선호됩니다.

주요 평가 지표 정리:

지표	이름	수식 개념	특징
MAE	평균 절대 오차	오차의 절대값 평균	직관적

지표	이름	수식 개념	특징
MSE	평균 제곱 오차	오차의 제곱 평균	큰 오차에 민감
RMSE	평균 제곱근 오차	MSE 에 루트	가장 일반적, 단위가 원래 단위와 동일
R^2	결정 계수	SSR / SST	0~1 사이, 1 에 가까울수록 좋음

RMSE 는 MSE 에 루트를 씌운 것입니다. MSE 는 제곱 때문에 단위가 원래 단위의 제곱이 됩니다 (예: 달러^2). 루트를 씌우면 원래 단위 (달러) 로 돌아옵니다. 그래서 RMSE 가 **가장 일반적으로 사용**됩니다.

R^2 (결정 계수) 는 독립변수가 종속변수를 **얼마나 잘 설명하는지의 비율**입니다. 0 에서 1 사이의 값을 가지며, **1 에 가까울수록 좋습니다**. 예를 들어 R^2 = 0.74 라면, 독립변수가 종속변수의 변동을 74% 설명한다는 뜻입니다.

자, 이제 코드로 계산해봅시다. 새 셀에 다음을 입력하세요.

```
from sklearn.metrics import mean_squared_error
mean_squared_error(y_test, pred) ** 0.5  # RMSE 계산
```

`mean_squared_error(y_test, pred)` 는 MSE 를 계산합니다. 여기에 `** 0.5`(0.5 제곱 = 루트) 를 붙이면 RMSE 가 됩니다.

결과: 약 **5684.93**

이 값은"평균적으로 약 5685 정도의 오차가 있다"는 뜻입니다. 보험료 단위 (달러) 로 약 5685 달러 정도 틀린다는 것이죠.

같은 결과를 더 간단하게 얻을 수도 있습니다.

```
mean_squared_error(y_test, pred, squared=False)  # 같은 결과
```

`squared=False` 를 넣으면 MSE 대신 RMSE 를 바로 계산해줍니다. 같은 약 5684.93 이 나올 것입니다.

이제 R^2 값도 확인합시다.

```
model.score(X_train, y_train)  # R² 계산
```

결과: 약 **0.7368**

R^2 = 0.7368 은 독립변수 (age, sex, bmi, children, smoker) 가 종속변수 (charges) 의 변동을 **약 73.7% 설명**한다는 뜻입니다. 나머지 26.3% 는 이 모델로 설명하지 못하는 부분입니다. 나쁘지 않지만, 아직 개선의 여지가 있다는 뜻이기도 합니다.

4.8 이해하기: 선형 회귀 (약 12 분)

지금까지 실습을 통해"사용법"을 배웠습니다. 이제 **원리**를 이해해봅시다.

선형 회귀의 수학적 원리

중학교 때 배운 일차 함수를 기억하시나요?

$$\mathbf{y = ax + b}$$

여기서 a 는 **기울기 (slope)**, b 는 **y 절편 (intercept)** 입니다.

선형 회귀가 하는 일은 바로 이것입니다. 데이터에 가장 잘 맞는 **직선**을 찾는 것입니다. 독립변수가 여러 개면 다음과 같이 확장됩니다.

$$y = a1x1 + a2x2 + a3*x3 + ...+ b$$

우리 문제에서는:

$$charges = a1age + a2sex + a3bmi + a4children + a5*smoker + b$$

모델이 학습한다는 것은 이 수식에서 **a1, a2, a3, a4, a5, b 값을 찾는 것**입니다.

손실 함수 (Loss Function)

그런데"가장 잘 맞는 직선"이란 무엇일까요? 이것을 수학적으로 정의해야 합니다.

데이터 점들과 직선 사이의 **거리 (오차) 의 합**이 가장 작은 직선이"가장 잘 맞는 직선"입니다. 구체적으로는, 오차의 제곱의 합을 최소화합니다. 이것을 **손실 함수 (Loss Function)** 또는 **비용 함수 (Cost Function)** 라고 합니다.

선형 회귀에서는 이 최소화 문제를 수학적으로 정확하게 풀 수 있습니다 (최소제곱법, Ordinary Least Squares). 그래서 학습 속도가 빠른 것입니다.

SST, SSR, SSE

R^2 를 이해하기 위해 세 가지 개념을 알아야 합니다.

- **SST (Total Sum of Squares)**: 전체 변동. "데이터가 평균으로부터 얼마나 흩어져 있는가?"
- **SSR (Regression Sum of Squares)**: 회귀로 설명되는 변동. "모델이 설명하는 부분"
- **SSE (Error Sum of Squares)**: 오차 변동. "모델이 설명하지 못하는 부분"

관계식: **SST = SSR + SSE**

그리고 **R^2 = SSR / SST** 입니다.

R^2 가 1 이면 SSR = SST, 즉 모델이 모든 변동을 설명한다는 뜻입니다. R^2 가 0 이면 모델이 아무것도 설명하지 못한다는 뜻입니다.

계수 (Coefficient) 확인

이제 모델이 학습한 계수 (기울기) 와 y 절편을 확인해봅시다. 새 셀에 다음을 입력하세요.

```
model.coef_  # 계수 확인
```

이것은 배열 (array) 로 나와서 어떤 변수의 계수인지 알기 어렵습니다. 보기 좋게 정리합시다.

```
pd.Series(model.coef_, index=X.columns)
```

결과:

```
age         264.799803
sex          17.344661
bmi         297.514806
children    469.339602
smoker    23469.280173
```

y 절편도 확인합시다.

```
model.intercept_  # y 절편
```

결과: **-11576.999976112367**

완성된 수식

이제 모델이 학습한 완전한 수식을 쓸 수 있습니다.

> **charges = 264.8 x age + 17.3 x sex + 297.5 x bmi + 469.3 x children + 23469.3 x smoker - 11577.0**

계수의 해석

이 수식의 각 계수가 의미하는 바를 해석해봅시다. 이것이 선형 회귀의 가장 큰 장점입니다.

- **age 의 계수 264.8**: 다른 변수가 동일할 때, 나이가 1 살 증가하면 보험료가 약 **265 달러 증가**합니다.
- **sex 의 계수 17.3**: 다른 변수가 동일할 때, 남성 (1) 이 여성 (0) 보다 보험료가 약 **17 달러 높습니다**. (차이가 매우 작습니다)
- **bmi 의 계수 297.5**: 다른 변수가 동일할 때, BMI 가 1 증가하면 보험료가 약 **298 달러 증가**합니다.
- **children 의 계수 469.3**: 다른 변수가 동일할 때, 자녀가 1 명 증가하면 보험료가 약 **469 달러 증가**합니다.
- **smoker 의 계수 23469.3**: 다른 변수가 동일할 때, 흡연자 (1) 는 비흡연자 (0) 보다 보험료가 약 **23,469 달러 높습니다**!

보세요, **smoker 의 계수가 압도적으로 큽니다.** 흡연 여부가 보험료에 가장 큰 영향을 미치는 변수라는 것을 알 수 있습니다. 이것은 직관적으로도 이해가 됩니다. 흡연은 건강에 큰 영향을 미치고, 따라서 보험사에서 흡연자에게 더 높은 보험료를 청구하는 것입니다.

반면, sex 의 계수는 17.3 으로 매우 작습니다. 성별은 보험료에 거의 영향을 미치지 않는다는 뜻입니다.

이런 식으로 각 변수의 영향력을 해석할 수 있는 것이 선형 회귀의 큰 장점입니다.

학습 마무리 (약 6 분)

전체 파이프라인 요약

오늘 배운 전체 과정을 요약하겠습니다. 이 과정은 앞으로 배울 모든 머신러닝 모델에서 반복됩니다.

```
4.1 문제 정의      → 무엇을 예측할 것인가?
    ↓
4.2 데이터 불러오기 → pandas로 CSV 파일 읽기
    ↓
4.3 데이터 확인     → head(), info(), describe()로 탐색
    ↓
4.4 전처리         → X/y 분리, train/test 분할
    ↓
4.5 모델링         → import → 객체 생성 → fit()
    ↓
4.6 예측           → predict()
    ↓
```

4.7 평가 → 테이블, 그래프, RMSE/R^2

핵심 개념 정리

개념	설명
선형 회귀	데이터의 관계를 직선으로 모델링하여 연속형 변수를 예측
독립변수 (X)	예측에 사용하는 입력 변수
종속변수 (y)	예측하려는 목표 변수
학습셋 / 시험셋	오버피팅 방지를 위해 데이터를 분리
fit()	모델 학습
predict()	모델로 예측
RMSE	평균 제곱근 오차, 예측 오차의 대표값
R^2	결정 계수, 모델의 설명력 (0~1)
계수 (coef_)	각 독립변수가 종속변수에 미치는 영향의 크기와 방향
절편 (intercept_)	모든 독립변수가 0 일 때의 기본 예측값

오늘의 모델 성능 요약

지표	값	해석
RMSE	약 5684.93	평균적으로 약 5685 달러의 예측 오차
R^2	약 0.7368	독립변수가 종속변수 변동의 73.7% 를 설명

R^2 가 0.74 정도이므로, 이 모델은"적당한"수준입니다. 더 좋은 성능을 원한다면 어떻게 해야 할까요?

1. **더 많은 독립변수**를 추가할 수 있습니다 (예: 거주 지역, 직업 등)
2. **비선형 모델**을 사용할 수 있습니다 (다음 장에서 배울 모델들)
3. **피처 엔지니어링**을 할 수 있습니다 (변수 변환, 교차항 추가 등)

이것들은 앞으로 차차 배우게 됩니다.

전체 코드 요약

마지막으로, 오늘 작성한 코드를 한눈에 정리하겠습니다.

```
# 1. 라이브러리 및 데이터 불러오기
import pandas as pd
file_url = 'https://media.githubusercontent.com/media/musthave-ML10/data_source/main/insurance
data = pd.read_csv(file_url)

# 2. 데이터 확인
data.head()
data.info()
round(data.describe(), 2)

# 3. 전처리 (데이터 분할)
X = data[['age', 'sex', 'bmi', 'children', 'smoker']]
y = data['charges']
```

```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=100)

# 4. 모델링
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)

# 5. 예측
pred = model.predict(X_test)

# 6. 평가
comparison = pd.DataFrame({'actual': y_test, 'pred': pred})
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(10,10))
sns.scatterplot(x='actual', y='pred', data=comparison)

from sklearn.metrics import mean_squared_error
print('RMSE:', mean_squared_error(y_test, pred, squared=False))
print('R²:', model.score(X_train, y_train))

# 7. 모델 해석
pd.Series(model.coef_, index=X.columns)
print('절편:', model.intercept_)
```

수고하셨습니다. 다음 시간에는 5 장에서 새로운 알고리즘을 배우겠습니다. 감사합니다.

강의록 작성: 2026 년 3 월 **기반 교재**: #MUSTHAVE 머신러닝 (골든래빗) 4 장